Neuro-Symbolic Reinforcement Learning
for Safe Autonomous Highway Driving

LTL-Shielded Deep Q-Networks on HighwayEnv

Prateek Ganguli

April 2026

Application: Autonomous Highway Driving

Setting

- 4-lane highway with **50 surrounding vehicles**
- Dense, stochastic traffic: vehicles spawn, merge, and cut in
- Ego agent must act in real-time from a partial observation

The Core Problem

- **Performance:** drive fast, stay in the rightmost lane
- **Safety:** never collide with another vehicle
- *Neural RL ignores safety; pure rules sacrifice performance.*

Simulator: HighwayEnv (Leurent 2018), kinematics-based, discrete action space

Ego (blue) navigating dense highway traffic

Application: Autonomous Highway Driving

Setting

- 4-lane highway with **50 surrounding vehicles**
- Dense, stochastic traffic: vehicles spawn, merge, and cut in
- Ego agent must act in real-time from a partial observation

The Core Problem

- **Performance:** drive fast, stay in the rightmost lane
- **Safety:** never collide with another vehicle
- *Neural RL ignores safety; pure rules sacrifice performance.*

Simulator: HighwayEnv (Leurent 2018), kinematics-based, discrete action space

Ego (blue) navigating dense highway traffic

Application: Autonomous Highway Driving

Setting

- 4-lane highway with **50 surrounding vehicles**
- Dense, stochastic traffic: vehicles spawn, merge, and cut in
- Ego agent must act in real-time from a partial observation

Ego (blue) navigating dense highway traffic

The Core Problem

- **Performance:** drive fast, stay in the rightmost lane
- **Safety:** never collide with another vehicle
- *Neural RL ignores safety; pure rules sacrifice performance.*

Simulator: HighwayEnv (Leurent 2018), kinematics-based, discrete action space

Application: Autonomous Highway Driving

Setting

- 4-lane highway with **50 surrounding vehicles**
- Dense, stochastic traffic: vehicles spawn, merge, and cut in
- Ego agent must act in real-time from a partial observation

Ego (blue) navigating dense highway traffic

The Core Problem

- **Performance:** drive fast, stay in the rightmost lane
- **Safety:** never collide with another vehicle
- *Neural RL ignores safety; pure rules sacrifice performance.*

Simulator: HighwayEnv (Leurent 2018), kinematics-based, discrete action space

Application: Autonomous Highway Driving

Setting

- 4-lane highway with **50 surrounding vehicles**
- Dense, stochastic traffic: vehicles spawn, merge, and cut in
- Ego agent must act in real-time from a partial observation

The Core Problem

- **Performance:** drive fast, stay in the rightmost lane
- **Safety:** never collide with another vehicle
- *Neural RL ignores safety; pure rules sacrifice performance.*

Simulator: HighwayEnv (Leurent 2018), kinematics-based, discrete action space

Ego (blue) navigating dense highway traffic

Application: Autonomous Highway Driving

Setting

- 4-lane highway with **50 surrounding vehicles**
- Dense, stochastic traffic: vehicles spawn, merge, and cut in
- Ego agent must act in real-time from a partial observation

The Core Problem

- **Performance:** drive fast, stay in the rightmost lane
- **Safety:** never collide with another vehicle
- *Neural RL ignores safety; pure rules sacrifice performance.*

Simulator: HighwayEnv (Leurent 2018), kinematics-based, discrete action space

Ego (blue) navigating dense highway traffic

Application: Autonomous Highway Driving

Setting

- 4-lane highway with **50 surrounding vehicles**
- Dense, stochastic traffic: vehicles spawn, merge, and cut in
- Ego agent must act in real-time from a partial observation

The Core Problem

- **Performance:** drive fast, stay in the rightmost lane
- **Safety:** never collide with another vehicle
- *Neural RL ignores safety; pure rules sacrifice performance.*

Simulator: HighwayEnv (Leurent 2018), kinematics-based, discrete action space

Ego (blue) navigating dense highway traffic

Application: Autonomous Highway Driving

Setting

- 4-lane highway with **50 surrounding vehicles**
- Dense, stochastic traffic: vehicles spawn, merge, and cut in
- Ego agent must act in real-time from a partial observation

The Core Problem

- **Performance:** drive fast, stay in the rightmost lane
- **Safety:** never collide with another vehicle
- *Neural RL ignores safety; pure rules sacrifice performance.*

Simulator: HighwayEnv (Leurent 2018), kinematics-based, discrete action space

Ego (blue) navigating dense highway traffic

Application: Autonomous Highway Driving

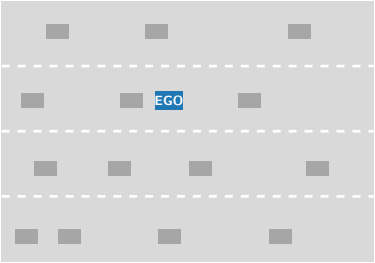
Setting

- 4-lane highway with **50 surrounding vehicles**
- Dense, stochastic traffic: vehicles spawn, merge, and cut in
- Ego agent must act in real-time from a partial observation

The Core Problem

- **Performance:** drive fast, stay in the rightmost lane
- **Safety:** never collide with another vehicle
- *Neural RL ignores safety; pure rules sacrifice performance.*

Simulator: HighwayEnv (Leurent 2018), kinematics-based, discrete action space

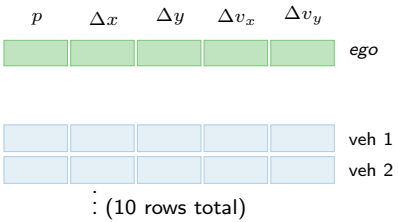


Ego (blue) navigating dense highway traffic

Input / Output Specification

Input: Ego-Relative Observation

$\mathbf{o}_t \in \mathbb{R}^{10\times 5}$ (10 nearest vehicles, 5 features each):



- $\Delta x, \Delta y \in [-100, 100]$ m; $\Delta v \in [-30, 30]$ m/s (normalized)
- Sorted by longitudinal distance; rear vehicles included
- Row order shifts each step, making the raw input **permutation-variant**

Output: 5 Discrete Actions

ID	Action
0	LANE_LEFT
1	IDLE
2	LANE_RIGHT
3	FASTER
4	SLOWER

Reward Function

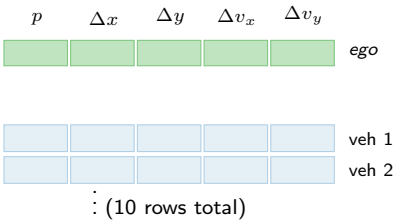
$$r_t = \underbrace{0.4\, r_{\text{speed}}}_{\text{go fast}} + \underbrace{0.1\, r_{\text{lane}}}_{\text{stay right}} - \underbrace{\mathbf{1}_{\text{crash}}}_{\text{survive}}$$

$r_{\text{speed}} \in [0, 1]$ linearly from 20 to 30 m/s; crash ends the episode.

Input / Output Specification

Input: Ego-Relative Observation

$\mathbf{o}_t \in \mathbb{R}^{10\times 5}$ (10 nearest vehicles, 5 features each):



- $\Delta x, \Delta y \in [-100, 100]$ m; $\Delta v \in [-30, 30]$ m/s (normalized)
- Sorted by longitudinal distance; rear vehicles included
- Row order shifts each step, making the raw input **permutation-variant**

Output: 5 Discrete Actions

ID	Action
0	LANE_LEFT
1	IDLE
2	LANE_RIGHT
3	FASTER
4	SLOWER

Reward Function

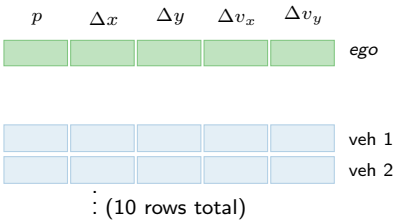
$$r_t = \underbrace{0.4\, r_{\text{speed}}}_{\text{go fast}} + \underbrace{0.1\, r_{\text{lane}}}_{\text{stay right}} - \underbrace{\mathbf{1}_{\text{crash}}}_{\text{survive}}$$

$r_{\text{speed}} \in [0, 1]$ linearly from 20 to 30 m/s; crash ends the episode.

Input / Output Specification

Input: Ego-Relative Observation

$\mathbf{o}_t \in \mathbb{R}^{10\times 5}$ (10 nearest vehicles, 5 features each):



- $\Delta x, \Delta y \in [-100, 100]$ m; $\Delta v \in [-30, 30]$ m/s (normalized)
- Sorted by longitudinal distance; rear vehicles included
- Row order shifts each step, making the raw input **permutation-variant**

Output: 5 Discrete Actions

ID	Action
0	LANE_LEFT
1	IDLE
2	LANE_RIGHT
3	FASTER
4	SLOWER

Reward Function

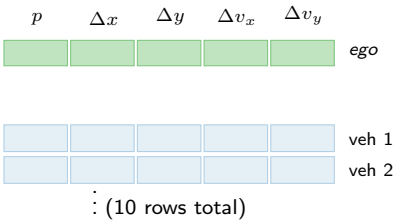
$$r_t = \underbrace{0.4\, r_{\text{speed}}}_{\text{go fast}} + \underbrace{0.1\, r_{\text{lane}}}_{\text{stay right}} - \underbrace{\mathbf{1}_{\text{crash}}}_{\text{survive}}$$

$r_{\text{speed}} \in [0, 1]$ linearly from 20 to 30 m/s; crash ends the episode.

Input / Output Specification

Input: Ego-Relative Observation

$\mathbf{o}_t \in \mathbb{R}^{10\times 5}$ (10 nearest vehicles, 5 features each):



- $\Delta x, \Delta y \in [-100, 100]$ m; $\Delta v \in [-30, 30]$ m/s (normalized)
- Sorted by longitudinal distance; rear vehicles included
- Row order shifts each step, making the raw input **permutation-variant**

Output: 5 Discrete Actions

ID	Action
0	LANE_LEFT
1	IDLE
2	LANE_RIGHT
3	FASTER
4	SLOWER

Reward Function

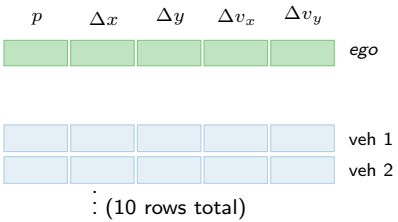
$$r_t = \underbrace{0.4\, r_{\text{speed}}}_{\text{go fast}} + \underbrace{0.1\, r_{\text{lane}}}_{\text{stay right}} - \underbrace{\mathbf{1}_{\text{crash}}}_{\text{survive}}$$

$r_{\text{speed}} \in [0, 1]$ linearly from 20 to 30 m/s; crash ends the episode.

Input / Output Specification

Input: Ego-Relative Observation

$\mathbf{o}_t \in \mathbb{R}^{10\times 5}$ (10 nearest vehicles, 5 features each):



- $\Delta x, \Delta y \in [-100, 100]$ m; $\Delta v \in [-30, 30]$ m/s (normalized)
- Sorted by longitudinal distance; rear vehicles included
- Row order shifts each step, making the raw input **permutation-variant**

Output: 5 Discrete Actions

ID	Action
0	LANE_LEFT
1	IDLE
2	LANE_RIGHT
3	FASTER
4	SLOWER

Reward Function

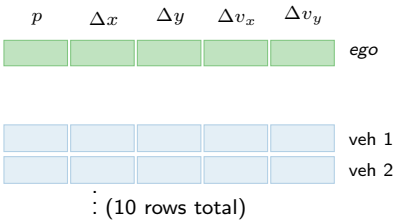
$$r_t = \underbrace{0.4\, r_{\text{speed}}}_{\text{go fast}} + \underbrace{0.1\, r_{\text{lane}}}_{\text{stay right}} - \underbrace{\mathbf{1}_{\text{crash}}}_{\text{survive}}$$

$r_{\text{speed}} \in [0, 1]$ linearly from 20 to 30 m/s; crash ends the episode.

Input / Output Specification

Input: Ego-Relative Observation

$\mathbf{o}_t \in \mathbb{R}^{10\times 5}$ (10 nearest vehicles, 5 features each):



- $\Delta x, \Delta y \in [-100, 100]$ m; $\Delta v \in [-30, 30]$ m/s (normalized)
- Sorted by longitudinal distance; rear vehicles included
- Row order shifts each step, making the raw input **permutation-variant**

Output: 5 Discrete Actions

ID	Action
0	LANE_LEFT
1	IDLE
2	LANE_RIGHT
3	FASTER
4	SLOWER

Reward Function

$$r_t = \underbrace{0.4\, r_{\text{speed}}}_{\text{go fast}} + \underbrace{0.1\, r_{\text{lane}}}_{\text{stay right}} - \underbrace{\mathbf{1}_{\text{crash}}}_{\text{survive}}$$

$r_{\text{speed}} \in [0, 1]$ linearly from 20 to 30 m/s; crash ends the episode.

Success Metrics & Formal Safety Specification

What counts as success?

100 independent test episodes:

- ↑ Mean reward (did the agent drive well?)
- ↓ Crash rate (did it stay safe?)
- ↓ Reward std (is the behavior consistent?)

Formal Safety Specification (LTL)

Four constraints, all enforced *at every timestep*:

- φ_1: $\mathbf{G}(\Delta x_{\text{front}} > 20\ \text{m})$ (no tailgating)
- φ_2: $\mathbf{G}(\text{TTC} > 2.0\ \text{s})$ (safe time-to-collision)
- φ_3: $\mathbf{G}(\text{LaneLeft} \Rightarrow \neg\text{left_occ})$
- φ_4: $\mathbf{G}(\text{LaneRight} \Rightarrow \neg\text{right_occ})$

These become the symbolic component of our solution.

Success Metrics & Formal Safety Specification

What counts as success?

100 independent test episodes:

- ↑ Mean reward (did the agent drive well?)
- ↓ Crash rate (did it stay safe?)
- ↓ Reward std (is the behavior consistent?)

Formal Safety Specification (LTL)

Four constraints, all enforced *at every timestep*:

- φ_1: $\mathbf{G}(\Delta x_{\text{front}} > 20 \text{ m})$ (no tailgating)
- φ_2: $\mathbf{G}(\text{TTC} > 2.0 \text{ s})$ (safe time-to-collision)
- φ_3: $\mathbf{G}(\text{LaneLeft} \Rightarrow \neg\text{left_occ})$
- φ_4: $\mathbf{G}(\text{LaneRight} \Rightarrow \neg\text{right_occ})$

These become the symbolic component of our solution.

Success Metrics & Formal Safety Specification

What counts as success?

100 independent test episodes:

- ↑ Mean reward (did the agent drive well?)
- ↓ Crash rate (did it stay safe?)
- ↓ Reward std (is the behavior consistent?)

Formal Safety Specification (LTL)

Four constraints, all enforced *at every timestep*:

- φ_1: $\mathbf{G}(\Delta x_{\text{front}} > 20 \text{ m})$ (no tailgating)
- φ_2: $\mathbf{G}(\text{TTC} > 2.0 \text{ s})$ (safe time-to-collision)
- φ_3: $\mathbf{G}(\text{LaneLeft} \Rightarrow \neg\text{left_occ})$
- φ_4: $\mathbf{G}(\text{LaneRight} \Rightarrow \neg\text{right_occ})$

These become the symbolic component of our solution.

Success Metrics & Formal Safety Specification

What counts as success?

100 independent test episodes:

- ↑ Mean reward (did the agent drive well?)
- ↓ Crash rate (did it stay safe?)
- ↓ Reward std (is the behavior consistent?)

Formal Safety Specification (LTL)

Four constraints, all enforced *at every timestep*:

- φ_1: $\mathbf{G}(\Delta x_{\text{front}} > 20 \text{ m})$ (no tailgating)
- φ_2: $\mathbf{G}(\text{TTC} > 2.0 \text{ s})$ (safe time-to-collision)
- φ_3: $\mathbf{G}(\text{LaneLeft} \Rightarrow \neg\text{left_occ})$
- φ_4: $\mathbf{G}(\text{LaneRight} \Rightarrow \neg\text{right_occ})$

These become the symbolic component of our solution.

Success Metrics & Formal Safety Specification

What counts as success?

100 independent test episodes:

- ↑ Mean reward (did the agent drive well?)
- ↓ Crash rate (did it stay safe?)
- ↓ Reward std (is the behavior consistent?)

Formal Safety Specification (LTL)

Four constraints, all enforced *at every timestep*:

- φ_1: $\mathbf{G}(\Delta x_{\text{front}} > 20 \text{ m})$ (no tailgating)
- φ_2: $\mathbf{G}(\text{TTC} > 2.0 \text{ s})$ (safe time-to-collision)
- φ_3: $\mathbf{G}(\text{LaneLeft} \Rightarrow \neg\text{left_occ})$
- φ_4: $\mathbf{G}(\text{LaneRight} \Rightarrow \neg\text{right_occ})$

These become the symbolic component of our solution.

Success Metrics & Formal Safety Specification

What counts as success?

100 independent test episodes:

- ↑ Mean reward (did the agent drive well?)
- ↓ Crash rate (did it stay safe?)
- ↓ Reward std (is the behavior consistent?)

Formal Safety Specification (LTL)

Four constraints, all enforced *at every timestep*:

- φ_1: $\mathbf{G}(\Delta x_{\text{front}} > 20 \text{ m})$ (no tailgating)
- φ_2: $\mathbf{G}(\text{TTC} > 2.0 \text{ s})$ (safe time-to-collision)
- φ_3: $\mathbf{G}(\text{LaneLeft} \Rightarrow \neg\text{left_occ})$
- φ_4: $\mathbf{G}(\text{LaneRight} \Rightarrow \neg\text{right_occ})$

These become the symbolic component of our solution.

Success Metrics & Formal Safety Specification

What counts as success?

100 independent test episodes:

- ↑ Mean reward (did the agent drive well?)
- ↓ Crash rate (did it stay safe?)
- ↓ Reward std (is the behavior consistent?)

Formal Safety Specification (LTL)

Four constraints, all enforced *at every timestep*:

- φ_1: $\mathbf{G}(\Delta x_{\text{front}} > 20 \text{ m})$ (no tailgating)
- φ_2: $\mathbf{G}(\text{TTC} > 2.0 \text{ s})$ (safe time-to-collision)
- φ_3: $\mathbf{G}(\text{LaneLeft} \Rightarrow \neg\text{left_occ})$
- φ_4: $\mathbf{G}(\text{LaneRight} \Rightarrow \neg\text{right_occ})$

These become the symbolic component of our solution.

Solution: LTL-Shielded Pearl DQN

Environment
$\mathbf{o}_t \in \mathbb{R}^{10\times 5}$

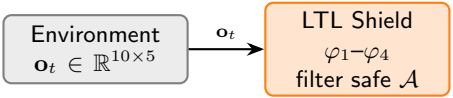
Key Design Decision

Shield acts *inside* `env.step()`, not post-hoc. Bellman update therefore uses only safe next-state actions:
$y = r + \gamma \max_{a' \in \mathcal{A}_{\text{safe}}(s')} Q_{\bar{\theta}}(s', a')$

Three Agents Compared

Pure Neural (Pearl DQN, no constraints)
Pure Symbolic (hand-coded FSM, 4 states)
Neuro-Symbolic (Shielded Pearl DQN)

Solution: LTL-Shielded Pearl DQN



Key Design Decision

Shield acts *inside* `env.step()`, not post-hoc. Bellman update therefore uses only safe next-state actions:

$y = r + \gamma \max_{a' \in \mathcal{A}_{\text{safe}}(s')} Q_{\bar{\theta}}(s', a')$

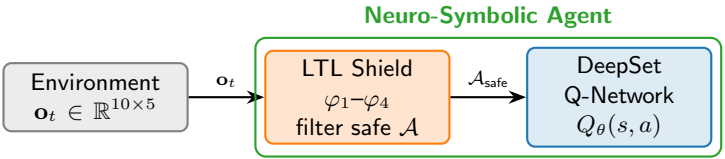
Three Agents Compared

Pure Neural (Pearl DQN, no constraints)

Pure Symbolic (hand-coded FSM, 4 states)

Neuro-Symbolic (Shielded Pearl DQN)

Solution: LTL-Shielded Pearl DQN



Key Design Decision

Shield acts *inside* `env.step()`, not post-hoc. Bellman update therefore uses only safe next-state actions:

$y = r + \gamma \max_{a' \in \mathcal{A}_{\text{safe}}(s')} Q_{\bar{\theta}}(s', a')$

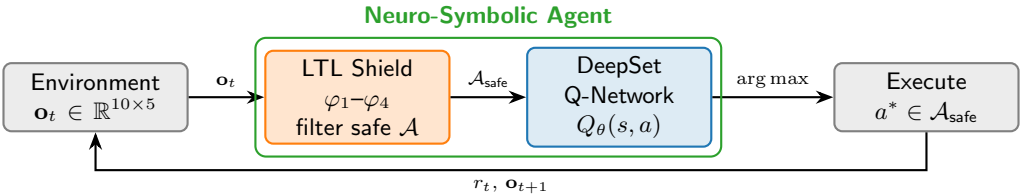
Three Agents Compared

Pure Neural (Pearl DQN, no constraints)

Pure Symbolic (hand-coded FSM, 4 states)

Neuro-Symbolic (Shielded Pearl DQN)

Solution: LTL-Shielded Pearl DQN



Key Design Decision

Shield acts *inside* `env.step()`, not post-hoc. Bellman update therefore uses only safe next-state actions:

$y = r + \gamma \max_{a' \in \mathcal{A}_{\text{safe}}(s')} Q_{\bar{\theta}}(s', a')$

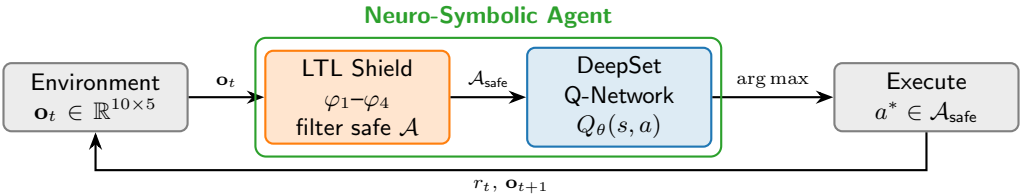
Three Agents Compared

Pure Neural (Pearl DQN, no constraints)

Pure Symbolic (hand-coded FSM, 4 states)

Neuro-Symbolic (Shielded Pearl DQN)

Solution: LTL-Shielded Pearl DQN



Key Design Decision

Shield acts *inside* `env.step()`, not post-hoc. Bellman update therefore uses only safe next-state actions:

$y = r + \gamma \max_{a' \in \mathcal{A}_{\text{safe}}(s')} Q_{\bar{\theta}}(s', a')$

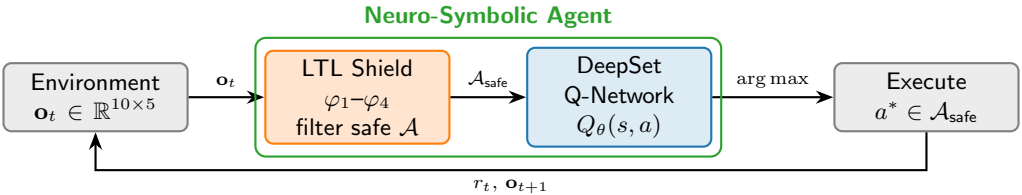
Three Agents Compared

Pure Neural (Pearl DQN, no constraints)

Pure Symbolic (hand-coded FSM, 4 states)

Neuro-Symbolic (Shielded Pearl DQN)

Solution: LTL-Shielded Pearl DQN



Key Design Decision

Shield acts *inside* `env.step()`, not post-hoc. Bellman update therefore uses only safe next-state actions:

$y = r + \gamma \max_{a' \in \mathcal{A}_{\text{safe}}(s')} Q_{\bar{\theta}}(s', a')$

Three Agents Compared

Pure Neural (Pearl DQN, no constraints)

Pure Symbolic (hand-coded FSM, 4 states)

Neuro-Symbolic (Shielded Pearl DQN)

Neural Component: DeepSet Q-Network

Why a DeepSet?

Vehicle rows shift each timestep, making the raw input permutation-variant. DeepSet aggregation is **order-invariant by construction**:

$$Q(s,a) = \rho\Big(\sum_{i=1}^{10} \phi(\mathbf{v}_i)\Big)$$

- Encoder ϕ: shared MLP $[5 \to 64 \to 64]$ per vehicle
- Aggregation: element-wise sum (permutation-invariant)
- Head ρ: MLP $[69 \to 256 \to 256 \to 5]$

Training: Double DQN + PER

- Double Q-learning: separate target network reduces overestimation
- Prioritized replay (α=0.6, β:0.4$\to$1.0): focus on high-error transitions
- Shield penalty -0.05: incentivizes the policy to internalize safety

Buffer	15 000 transitions
LR	5×10^{-4}
γ	0.8
ε	$1.0 \to 0.05$ (100k steps)

Neural Component: DeepSet Q-Network

Why a DeepSet?

Vehicle rows shift each timestep, making the raw input permutation-variant. DeepSet aggregation is **order-invariant by construction**:

$$Q(s,a) = \rho\Big(\sum_{i=1}^{10} \phi(\mathbf{v}_i)\Big)$$

- Encoder ϕ: shared MLP $[5 \to 64 \to 64]$ per vehicle
- Aggregation: element-wise sum (permutation-invariant)
- Head ρ: MLP $[69 \to 256 \to 256 \to 5]$

Training: Double DQN + PER

- Double Q-learning: separate target network reduces overestimation
- Prioritized replay (α=0.6, β:0.4$\to$1.0): focus on high-error transitions
- Shield penalty -0.05: incentivizes the policy to internalize safety

Buffer	15 000 transitions
LR	5×10^{-4}
γ	0.8
ε	$1.0 \to 0.05$ (100k steps)

Neural Component: DeepSet Q-Network

Why a DeepSet?

Vehicle rows shift each timestep, making the raw input permutation-variant. DeepSet aggregation is **order-invariant by construction**:

$$Q(s,a) = \rho\Big(\sum_{i=1}^{10} \phi(\mathbf{v}_i)\Big)$$

- Encoder ϕ: shared MLP $[5 \to 64 \to 64]$ per vehicle
- Aggregation: element-wise sum (permutation-invariant)
- Head ρ: MLP $[69 \to 256 \to 256 \to 5]$

Training: Double DQN + PER

- Double Q-learning: separate target network reduces overestimation
- Prioritized replay (α=0.6, β:0.4→1.0): focus on high-error transitions
- Shield penalty -0.05: incentivizes the policy to internalize safety

Buffer	15 000 transitions
LR	5×10^{-4}
γ	0.8
ε	$1.0 \to 0.05$ (100k steps)

Neural Component: DeepSet Q-Network

Why a DeepSet?

Vehicle rows shift each timestep, making the raw input permutation-variant. DeepSet aggregation is **order-invariant by construction**:

$$Q(s,a) = \rho\Big(\sum_{i=1}^{10} \phi(\mathbf{v}_i)\Big)$$

- Encoder ϕ: shared MLP $[5 \to 64 \to 64]$ per vehicle
- Aggregation: element-wise sum (permutation-invariant)
- Head ρ: MLP $[69 \to 256 \to 256 \to 5]$

Training: Double DQN + PER

- Double Q-learning: separate target network reduces overestimation
- Prioritized replay (α=0.6, β:0.4→1.0): focus on high-error transitions
- Shield penalty -0.05: incentivizes the policy to internalize safety

Buffer	15 000 transitions
LR	5×10^{-4}
γ	0.8
ε	$1.0 \to 0.05$ (100k steps)

Neural Component: DeepSet Q-Network

Why a DeepSet?

Vehicle rows shift each timestep, making the raw input permutation-variant. DeepSet aggregation is **order-invariant by construction**:

$$Q(s,a) = \rho\Big(\sum_{i=1}^{10} \phi(\mathbf{v}_i)\Big)$$

- Encoder ϕ: shared MLP $[5 \to 64 \to 64]$ per vehicle
- Aggregation: element-wise sum (permutation-invariant)
- Head ρ: MLP $[69 \to 256 \to 256 \to 5]$

Training: Double DQN + PER

- Double Q-learning: separate target network reduces overestimation
- Prioritized replay (α=0.6, β:0.4→1.0): focus on high-error transitions
- Shield penalty -0.05: incentivizes the policy to internalize safety

Buffer	15 000 transitions
LR	5×10^{-4}
γ	0.8
ε	$1.0 \to 0.05$ (100k steps)

Neural Component: DeepSet Q-Network

Why a DeepSet?

Vehicle rows shift each timestep, making the raw input permutation-variant. DeepSet aggregation is **order-invariant by construction**:

$$Q(s,a) = \rho\Big(\sum_{i=1}^{10} \phi(\mathbf{v}_i)\Big)$$

- Encoder ϕ: shared MLP $[5 \to 64 \to 64]$ per vehicle
- Aggregation: element-wise sum (permutation-invariant)
- Head ρ: MLP $[69 \to 256 \to 256 \to 5]$

Training: Double DQN + PER

- Double Q-learning: separate target network reduces overestimation
- Prioritized replay (α=0.6, β:0.4→1.0): focus on high-error transitions
- Shield penalty -0.05: incentivizes the policy to internalize safety

Buffer	15 000 transitions
LR	5×10^{-4}
γ	0.8
ε	$1.0 \to 0.05$ (100k steps)

Symbolic Component: LTL Safety Shield

Four Globally-Enforced Predicates

- φ_1: No tailgating ($\Delta x_{\text{front}} > 20$ m)
- φ_2: Safe TTC (> 2.0 s)
- φ_3: Lane-left only if left lane is clear
- φ_4: Lane-right only if right lane is clear

Shield Evaluation (per step)

1. Decode $\mathbf{o}_t$ into per-vehicle predicates
2. For each candidate action, simulate one step symbolically
3. Retain a iff $\varphi_1 \wedge \varphi_2 \wedge \varphi_3 \wedge \varphi_4$ holds
4. Return $\mathcal{A}_{\text{safe}} \subseteq \{0, \ldots, 4\}$ to Q-network

Implementation

Built on **Meta's Pearl** framework. `LTLShieldSafetyModule` implements Pearl's `SafetyModule` ABC, making it modular and composable.

What this guarantees

- Every approved action satisfies $\varphi_1 \neg\varphi_4$ by construction
- Each veto is **human-readable**: *"φ_3 violated: left lane occupied"*
- No gradient computation; purely symbolic evaluation

Symbolic Component: LTL Safety Shield

Four Globally-Enforced Predicates

- φ_1: No tailgating ($\Delta x_{\text{front}} > 20$ m)
- φ_2: Safe TTC (> 2.0 s)
- φ_3: Lane-left only if left lane is clear
- φ_4: Lane-right only if right lane is clear

Shield Evaluation (per step)

1. Decode $\mathbf{o}_t$ into per-vehicle predicates
2. For each candidate action, simulate one step symbolically
3. Retain a iff $\varphi_1 \wedge \varphi_2 \wedge \varphi_3 \wedge \varphi_4$ holds
4. Return $\mathcal{A}_{\text{safe}} \subseteq \{0, \ldots, 4\}$ to Q-network

Implementation

Built on **Meta's Pearl** framework. `LTLShieldSafetyModule` implements Pearl's `SafetyModule` ABC, making it modular and composable.

What this guarantees

- Every approved action satisfies $\varphi_1 \neg \varphi_4$ by construction
- Each veto is **human-readable**: *"φ_3 violated: left lane occupied"*
- No gradient computation; purely symbolic evaluation

Symbolic Component: LTL Safety Shield

Four Globally-Enforced Predicates

- φ_1: No tailgating ($\Delta x_{\text{front}} > 20$ m)
- φ_2: Safe TTC (> 2.0 s)
- φ_3: Lane-left only if left lane is clear
- φ_4: Lane-right only if right lane is clear

Shield Evaluation (per step)

1. Decode $\mathbf{o}_t$ into per-vehicle predicates
2. For each candidate action, simulate one step symbolically
3. Retain a iff $\varphi_1 \wedge \varphi_2 \wedge \varphi_3 \wedge \varphi_4$ holds
4. Return $\mathcal{A}_{\text{safe}} \subseteq \{0, \ldots, 4\}$ to Q-network

Implementation

Built on **Meta's Pearl** framework. `LTLShieldSafetyModule` implements Pearl's `SafetyModule` ABC, making it modular and composable.

What this guarantees

- Every approved action satisfies $\varphi_1 \neg \varphi_4$ by construction
- Each veto is **human-readable**: *"φ_3 violated: left lane occupied"*
- No gradient computation; purely symbolic evaluation

Symbolic Component: LTL Safety Shield

Four Globally-Enforced Predicates

- φ_1: No tailgating ($\Delta x_{\text{front}} > 20$ m)
- φ_2: Safe TTC (> 2.0 s)
- φ_3: Lane-left only if left lane is clear
- φ_4: Lane-right only if right lane is clear

Shield Evaluation (per step)

1. Decode $\mathbf{o}_t$ into per-vehicle predicates
2. For each candidate action, simulate one step symbolically
3. Retain a iff $\varphi_1 \wedge \varphi_2 \wedge \varphi_3 \wedge \varphi_4$ holds
4. Return $\mathcal{A}_{\text{safe}} \subseteq \{0, \ldots, 4\}$ to Q-network

Implementation

Built on **Meta's Pearl** framework. `LTLShieldSafetyModule` implements Pearl's `SafetyModule` ABC, making it modular and composable.

What this guarantees

- Every approved action satisfies $\varphi_1 \neg \varphi_4$ by construction
- Each veto is **human-readable**: *"φ_3 violated: left lane occupied"*
- No gradient computation; purely symbolic evaluation

Symbolic Component: LTL Safety Shield

Four Globally-Enforced Predicates

- φ_1: No tailgating ($\Delta x_{\text{front}} > 20$ m)
- φ_2: Safe TTC (> 2.0 s)
- φ_3: Lane-left only if left lane is clear
- φ_4: Lane-right only if right lane is clear

Shield Evaluation (per step)

1. Decode $\mathbf{o}_t$ into per-vehicle predicates
2. For each candidate action, simulate one step symbolically
3. Retain a iff $\varphi_1 \wedge \varphi_2 \wedge \varphi_3 \wedge \varphi_4$ holds
4. Return $\mathcal{A}_{\text{safe}} \subseteq \{0, \ldots, 4\}$ to Q-network

Implementation

Built on **Meta's Pearl** framework. `LTLShieldSafetyModule` implements Pearl's `SafetyModule` ABC, making it modular and composable.

What this guarantees

- Every approved action satisfies $\varphi_1 \neg \varphi_4$ by construction
- Each veto is **human-readable**: *"φ_3 violated: left lane occupied"*
- No gradient computation; purely symbolic evaluation

Symbolic Component: LTL Safety Shield

Four Globally-Enforced Predicates

- φ_1: No tailgating ($\Delta x_{\text{front}} > 20$ m)
- φ_2: Safe TTC (> 2.0 s)
- φ_3: Lane-left only if left lane is clear
- φ_4: Lane-right only if right lane is clear

Shield Evaluation (per step)

1. Decode $\mathbf{o}_t$ into per-vehicle predicates
2. For each candidate action, simulate one step symbolically
3. Retain a iff $\varphi_1 \wedge \varphi_2 \wedge \varphi_3 \wedge \varphi_4$ holds
4. Return $\mathcal{A}_{\text{safe}} \subseteq \{0, \ldots, 4\}$ to Q-network

Implementation

Built on **Meta's Pearl** framework. `LTLShieldSafetyModule` implements Pearl's `SafetyModule` ABC, making it modular and composable.

What this guarantees

- Every approved action satisfies φ_1–φ_4 by construction
- Each veto is **human-readable**: *"φ_3 violated: left lane occupied"*
- No gradient computation; purely symbolic evaluation

Symbolic Component: LTL Safety Shield

Four Globally-Enforced Predicates

- φ_1: No tailgating ($\Delta x_{\text{front}} > 20$ m)
- φ_2: Safe TTC (> 2.0 s)
- φ_3: Lane-left only if left lane is clear
- φ_4: Lane-right only if right lane is clear

Shield Evaluation (per step)

1. Decode $\mathbf{o}_t$ into per-vehicle predicates
2. For each candidate action, simulate one step symbolically
3. Retain a iff $\varphi_1 \wedge \varphi_2 \wedge \varphi_3 \wedge \varphi_4$ holds
4. Return $\mathcal{A}_{\text{safe}} \subseteq \{0, \ldots, 4\}$ to Q-network

Implementation

Built on **Meta's Pearl** framework. `LTLShieldSafetyModule` implements Pearl's `SafetyModule` ABC, making it modular and composable.

What this guarantees

- Every approved action satisfies $\varphi_1 \neg \varphi_4$ by construction
- Each veto is **human-readable**: *"φ_3 violated: left lane occupied"*
- No gradient computation; purely symbolic evaluation

Symbolic Component: LTL Safety Shield

Four Globally-Enforced Predicates

- φ_1: No tailgating ($\Delta x_{\text{front}} > 20$ m)
- φ_2: Safe TTC (> 2.0 s)
- φ_3: Lane-left only if left lane is clear
- φ_4: Lane-right only if right lane is clear

Shield Evaluation (per step)

1. Decode $\mathbf{o}_t$ into per-vehicle predicates
2. For each candidate action, simulate one step symbolically
3. Retain a iff $\varphi_1 \wedge \varphi_2 \wedge \varphi_3 \wedge \varphi_4$ holds
4. Return $\mathcal{A}_{\text{safe}} \subseteq \{0, \ldots, 4\}$ to Q-network

Implementation

Built on **Meta's Pearl** framework. `LTLShieldSafetyModule` implements Pearl's `SafetyModule` ABC, making it modular and composable.

What this guarantees

- Every approved action satisfies $\varphi_1 \neg \varphi_4$ by construction
- Each veto is **human-readable**: *"φ_3 violated: left lane occupied"*
- No gradient computation; purely symbolic evaluation

Symbolic Component: LTL Safety Shield

Four Globally-Enforced Predicates

- φ_1: No tailgating ($\Delta x_{\text{front}} > 20$ m)
- φ_2: Safe TTC (> 2.0 s)
- φ_3: Lane-left only if left lane is clear
- φ_4: Lane-right only if right lane is clear

Shield Evaluation (per step)

1. Decode $\mathbf{o}_t$ into per-vehicle predicates
2. For each candidate action, simulate one step symbolically
3. Retain a iff $\varphi_1 \wedge \varphi_2 \wedge \varphi_3 \wedge \varphi_4$ holds
4. Return $\mathcal{A}_{\text{safe}} \subseteq \{0, \ldots, 4\}$ to Q-network

Implementation

Built on **Meta's Pearl** framework. `LTLShieldSafetyModule` implements Pearl's `SafetyModule` ABC, making it modular and composable.

What this guarantees

- Every approved action satisfies $\varphi_1 \neg \varphi_4$ by construction
- Each veto is **human-readable**: *"φ_3 violated: left lane occupied"*
- No gradient computation; purely symbolic evaluation

Symbolic Component: LTL Safety Shield

Four Globally-Enforced Predicates

- φ_1: No tailgating ($\Delta x_{\text{front}} > 20$ m)
- φ_2: Safe TTC (> 2.0 s)
- φ_3: Lane-left only if left lane is clear
- φ_4: Lane-right only if right lane is clear

Shield Evaluation (per step)

1. Decode $\mathbf{o}_t$ into per-vehicle predicates
2. For each candidate action, simulate one step symbolically
3. Retain a iff $\varphi_1 \wedge \varphi_2 \wedge \varphi_3 \wedge \varphi_4$ holds
4. Return $\mathcal{A}_{\text{safe}} \subseteq \{0, \ldots, 4\}$ to Q-network

Implementation

Built on **Meta's Pearl** framework. `LTLShieldSafetyModule` implements Pearl's `SafetyModule` ABC, making it modular and composable.

What this guarantees

- Every approved action satisfies $\varphi_1 \neg \varphi_4$ by construction
- Each veto is **human-readable**: *"φ_3 violated: left lane occupied"*
- No gradient computation; purely symbolic evaluation

Symbolic Component: LTL Safety Shield

Four Globally-Enforced Predicates

- φ_1: No tailgating ($\Delta x_{\text{front}} > 20$ m)
- φ_2: Safe TTC (> 2.0 s)
- φ_3: Lane-left only if left lane is clear
- φ_4: Lane-right only if right lane is clear

Shield Evaluation (per step)

1. Decode $\mathbf{o}_t$ into per-vehicle predicates
2. For each candidate action, simulate one step symbolically
3. Retain a iff $\varphi_1 \wedge \varphi_2 \wedge \varphi_3 \wedge \varphi_4$ holds
4. Return $\mathcal{A}_{\text{safe}} \subseteq \{0, \ldots, 4\}$ to Q-network

Implementation

Built on **Meta's Pearl** framework. `LTLShieldSafetyModule` implements Pearl's `SafetyModule` ABC, making it modular and composable.

What this guarantees

- Every approved action satisfies $\varphi_1 \neg \varphi_4$ by construction
- Each veto is **human-readable**: *"φ_3 violated: left lane occupied"*
- No gradient computation; purely symbolic evaluation

Symbolic Component: LTL Safety Shield

Four Globally-Enforced Predicates

- φ_1: No tailgating ($\Delta x_{\text{front}} > 20$ m)
- φ_2: Safe TTC (> 2.0 s)
- φ_3: Lane-left only if left lane is clear
- φ_4: Lane-right only if right lane is clear

Shield Evaluation (per step)

1. Decode $\mathbf{o}_t$ into per-vehicle predicates
2. For each candidate action, simulate one step symbolically
3. Retain a iff $\varphi_1 \wedge \varphi_2 \wedge \varphi_3 \wedge \varphi_4$ holds
4. Return $\mathcal{A}_{\text{safe}} \subseteq \{0, \ldots, 4\}$ to Q-network

Implementation

Built on **Meta's Pearl** framework. `LTLShieldSafetyModule` implements Pearl's `SafetyModule` ABC, making it modular and composable.

What this guarantees

- Every approved action satisfies φ_1–φ_4 by construction
- Each veto is **human-readable**: *"φ_3 violated: left lane occupied"*
- No gradient computation; purely symbolic evaluation

Symbolic Component: LTL Safety Shield

Four Globally-Enforced Predicates

- φ_1: No tailgating ($\Delta x_{\text{front}} > 20$ m)
- φ_2: Safe TTC (> 2.0 s)
- φ_3: Lane-left only if left lane is clear
- φ_4: Lane-right only if right lane is clear

Shield Evaluation (per step)

1. Decode $\mathbf{o}_t$ into per-vehicle predicates
2. For each candidate action, simulate one step symbolically
3. Retain a iff $\varphi_1 \wedge \varphi_2 \wedge \varphi_3 \wedge \varphi_4$ holds
4. Return $\mathcal{A}_{\text{safe}} \subseteq \{0, \ldots, 4\}$ to Q-network

Implementation

Built on **Meta's Pearl** framework. `LTLShieldSafetyModule` implements Pearl's `SafetyModule` ABC, making it modular and composable.

What this guarantees

- Every approved action satisfies φ_1–φ_4 by construction
- Each veto is **human-readable**: *"φ_3 violated: left lane occupied"*
- No gradient computation; purely symbolic evaluation

Symbolic Component: LTL Safety Shield

Four Globally-Enforced Predicates

- φ_1: No tailgating ($\Delta x_{\text{front}} > 20$ m)
- φ_2: Safe TTC (> 2.0 s)
- φ_3: Lane-left only if left lane is clear
- φ_4: Lane-right only if right lane is clear

Shield Evaluation (per step)

1. Decode $\mathbf{o}_t$ into per-vehicle predicates
2. For each candidate action, simulate one step symbolically
3. Retain a iff $\varphi_1 \wedge \varphi_2 \wedge \varphi_3 \wedge \varphi_4$ holds
4. Return $\mathcal{A}_{\text{safe}} \subseteq \{0, \ldots, 4\}$ to Q-network

Implementation

Built on **Meta's Pearl** framework. `LTLShieldSafetyModule` implements Pearl's `SafetyModule` ABC, making it modular and composable.

What this guarantees

- Every approved action satisfies φ_1–φ_4 by construction
- Each veto is **human-readable**: *"φ_3 violated: left lane occupied"*
- No gradient computation; purely symbolic evaluation

Neuro-Symbolic Checklist: The Two Parts

1. The Neuro Part

DeepSet Double-DQN with PER

- Learns a Q-function from raw interactions (100k steps)
- Discovers *when* to speed up, slow down, or change lane
- DeepSet encoder generalizes across any vehicle ordering
- Policy learns to respect the shield: override rate falls during training

2. The Symbolic Part

LTL Safety Shield (φ_1–φ_4)

- Encodes expert safety knowledge as inviolable logical rules
- Deterministic predicate evaluation; no training required
- Provides a **hard filter** on the Q-network's action choices
- If the shield approves a, no known unsafe outcome follows

Neuro-Symbolic Checklist: The Two Parts

1. The Neuro Part

DeepSet Double-DQN with PER

- Learns a Q-function from raw interactions (100k steps)
- Discovers *when* to speed up, slow down, or change lane
- DeepSet encoder generalizes across any vehicle ordering
- Policy learns to respect the shield: override rate falls during training

2. The Symbolic Part

LTL Safety Shield (φ_1–φ_4)

- Encodes expert safety knowledge as inviolable logical rules
- Deterministic predicate evaluation; no training required
- Provides a **hard filter** on the Q-network's action choices
- If the shield approves a, no known unsafe outcome follows

Neuro-Symbolic Checklist: The Two Parts

1. The Neuro Part

DeepSet Double-DQN with PER

- Learns a Q-function from raw interactions (100k steps)
- Discovers *when* to speed up, slow down, or change lane
- DeepSet encoder generalizes across any vehicle ordering
- Policy learns to respect the shield: override rate falls during training

2. The Symbolic Part

LTL Safety Shield (φ_1–φ_4)

- Encodes expert safety knowledge as inviolable logical rules
- Deterministic predicate evaluation; no training required
- Provides a **hard filter** on the Q-network's action choices
- If the shield approves a, no known unsafe outcome follows

Neuro-Symbolic Checklist: The Two Parts

1. The Neuro Part

DeepSet Double-DQN with PER

- Learns a Q-function from raw interactions (100k steps)
- Discovers *when* to speed up, slow down, or change lane
- DeepSet encoder generalizes across any vehicle ordering
- Policy learns to respect the shield: override rate falls during training

2. The Symbolic Part

LTL Safety Shield (φ_1–φ_4)

- Encodes expert safety knowledge as inviolable logical rules
- Deterministic predicate evaluation; no training required
- Provides a **hard filter** on the Q-network's action choices
- If the shield approves a, no known unsafe outcome follows

Neuro-Symbolic Checklist: The Two Parts

1. The Neuro Part

DeepSet Double-DQN with PER

- Learns a Q-function from raw interactions (100k steps)
- Discovers *when* to speed up, slow down, or change lane
- DeepSet encoder generalizes across any vehicle ordering
- Policy learns to respect the shield: override rate falls during training

2. The Symbolic Part

LTL Safety Shield (φ_1–φ_4)

- Encodes expert safety knowledge as inviolable logical rules
- Deterministic predicate evaluation; no training required
- Provides a **hard filter** on the Q-network's action choices
- If the shield approves a, no known unsafe outcome follows

Neuro-Symbolic Checklist: Generalization & Fragility

3. Generalization of the Neuro Part

- Permutation invariance: $\sum_i \phi(\mathbf{v}_i)$ is independent of vehicle ordering, handling any traffic configuration
- Reward varies continuously with speed, giving the network a smooth training signal
- Limitation: does not transfer to unseen action spaces or sensor modalities

4. Fragility of the Symbolic Part

- Coverage gaps: φ_1–φ_4 miss diagonal or merging-at-speed scenarios
- Sensor noise: noisy kinematics can flip predicate truth values
- Fixed thresholds: 20 m gap, 2 s TTC, conservative and not adaptive
- Spawn crashes: vehicles appearing adjacent bypass the 1-step lookahead

Neuro-Symbolic Checklist: Generalization & Fragility

3. Generalization of the Neuro Part

- Permutation invariance: $\sum_i \phi(\mathbf{v}_i)$ is independent of vehicle ordering, handling any traffic configuration
- Reward varies continuously with speed, giving the network a smooth training signal
- Limitation: does not transfer to unseen action spaces or sensor modalities

4. Fragility of the Symbolic Part

- Coverage gaps: φ_1–φ_4 miss diagonal or merging-at-speed scenarios
- Sensor noise: noisy kinematics can flip predicate truth values
- Fixed thresholds: 20 m gap, 2 s TTC, conservative and not adaptive
- Spawn crashes: vehicles appearing adjacent bypass the 1-step lookahead

Neuro-Symbolic Checklist: Generalization & Fragility

3. Generalization of the Neuro Part

- Permutation invariance: $\sum_i \phi(\mathbf{v}_i)$ is independent of vehicle ordering, handling any traffic configuration
- Reward varies continuously with speed, giving the network a smooth training signal
- Limitation: does not transfer to unseen action spaces or sensor modalities

4. Fragility of the Symbolic Part

- Coverage gaps: φ_1–φ_4 miss diagonal or merging-at-speed scenarios
- Sensor noise: noisy kinematics can flip predicate truth values
- Fixed thresholds: 20 m gap, 2 s TTC, conservative and not adaptive
- Spawn crashes: vehicles appearing adjacent bypass the 1-step lookahead

Neuro-Symbolic Checklist: Generalization & Fragility

3. Generalization of the Neuro Part

- Permutation invariance: $\sum_i \phi(\mathbf{v}_i)$ is independent of vehicle ordering, handling any traffic configuration
- Reward varies continuously with speed, giving the network a smooth training signal
- Limitation: does not transfer to unseen action spaces or sensor modalities

4. Fragility of the Symbolic Part

- Coverage gaps: φ_1–φ_4 miss diagonal or merging-at-speed scenarios
- Sensor noise: noisy kinematics can flip predicate truth values
- Fixed thresholds: 20 m gap, 2 s TTC, conservative and not adaptive
- Spawn crashes: vehicles appearing adjacent bypass the 1-step lookahead

Neuro-Symbolic Checklist: Generalization & Fragility

3. Generalization of the Neuro Part

- Permutation invariance: $\sum_i \phi(\mathbf{v}_i)$ is independent of vehicle ordering, handling any traffic configuration
- Reward varies continuously with speed, giving the network a smooth training signal
- Limitation: does not transfer to unseen action spaces or sensor modalities

4. Fragility of the Symbolic Part

- Coverage gaps: φ_1–φ_4 miss diagonal or merging-at-speed scenarios
- Sensor noise: noisy kinematics can flip predicate truth values
- Fixed thresholds: 20 m gap, 2 s TTC, conservative and not adaptive
- Spawn crashes: vehicles appearing adjacent bypass the 1-step lookahead

Neuro-Symbolic Checklist: Benefits

5. Benefits of the Combination

Safety & Explainability

Crash rate:

$\mathbf{27\% \rightarrow 18\%}$

Each shield veto has a human-readable reason (φ_i violated)

Consistency

Reward std:

$\mathbf{16.3 \rightarrow 11.4}$ (-30%)

Shield prevents catastrophic exploratory crashes early in training

Data Efficiency

Smaller safe action space yields cleaner, faster exploration

Override penalty steers the reward signal toward safe behavior

Combined Effect

After controlling for spawn crashes: FSM = 23.5%, Neural = 23.2%; *neither alone beats the other.*
Neuro-Symbolic achieves **18.0%**: the gain exceeds what either component produces alone.

Neuro-Symbolic Checklist: Benefits

5. Benefits of the Combination

Safety & Explainability

Crash rate:

27% → 18%

Each shield veto has a human-readable reason (φ_i violated)

Consistency

Reward std:

16.3 → 11.4 (−30%)

Shield prevents catastrophic exploratory crashes early in training

Data Efficiency

Smaller safe action space yields cleaner, faster exploration

Override penalty steers the reward signal toward safe behavior

Combined Effect

After controlling for spawn crashes: FSM = 23.5%, Neural = 23.2%; *neither alone beats the other.*
Neuro-Symbolic achieves **18.0%**: the gain exceeds what either component produces alone.

Neuro-Symbolic Checklist: Benefits

5. Benefits of the Combination

Safety & Explainability

Crash rate:

27% → 18%

Each shield veto has a human-readable reason (φ_i violated)

Consistency

Reward std:

16.3 → 11.4 (-30%)

Shield prevents catastrophic exploratory crashes early in training

Data Efficiency

Smaller safe action space yields cleaner, faster exploration

Override penalty steers the reward signal toward safe behavior

Combined Effect

After controlling for spawn crashes: FSM = 23.5%, Neural = 23.2%; *neither alone beats the other.*
Neuro-Symbolic achieves **18.0%**: the gain exceeds what either component produces alone.

Neuro-Symbolic Checklist: Benefits

5. Benefits of the Combination

Safety & Explainability

Crash rate:

27% → 18%

Each shield veto has a human-readable reason (φ_i violated)

Consistency

Reward std:

16.3 → 11.4 (-30%)

Shield prevents catastrophic exploratory crashes early in training

Data Efficiency

Smaller safe action space yields cleaner, faster exploration

Override penalty steers the reward signal toward safe behavior

Combined Effect

After controlling for spawn crashes: FSM = 23.5%, Neural = 23.2%; *neither alone beats the other.*
Neuro-Symbolic achieves **18.0%**: the gain exceeds what either component produces alone.

Neuro-Symbolic Checklist: Benefits

5. Benefits of the Combination

Safety & Explainability

Crash rate:

27% → 18%

Each shield veto has a human-readable reason (φ_i violated)

Consistency

Reward std:

16.3 → 11.4 (-30%)

Shield prevents catastrophic exploratory crashes early in training

Data Efficiency

Smaller safe action space yields cleaner, faster exploration

Override penalty steers the reward signal toward safe behavior

Combined Effect

After controlling for spawn crashes: FSM = 23.5%, Neural = 23.2%; *neither alone beats the other.*
Neuro-Symbolic achieves **18.0%**: the gain exceeds what either component produces alone.

Results: Training Dynamics

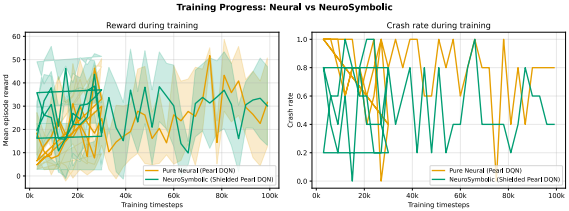


Figure: Training reward & crash rate vs. timestep (100k steps)

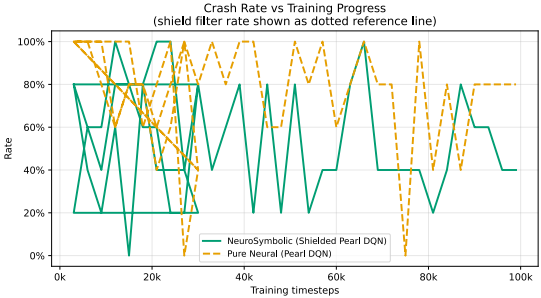


Figure: Shield override rate during training

- **Neural** reward rises faster, but unconstrained exploration yields more crashes
- Shield **override rate decreases** over training: policy learns to avoid vetoes
- Shielded agent reaches a **lower steady-state crash rate** despite slower initial reward growth

Results: Training Dynamics

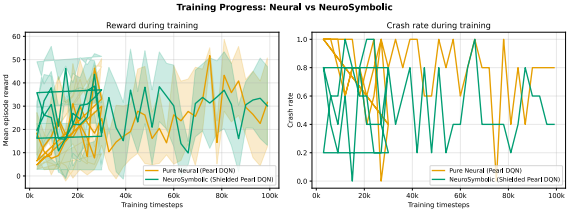


Figure: Training reward & crash rate vs. timestep (100k steps)

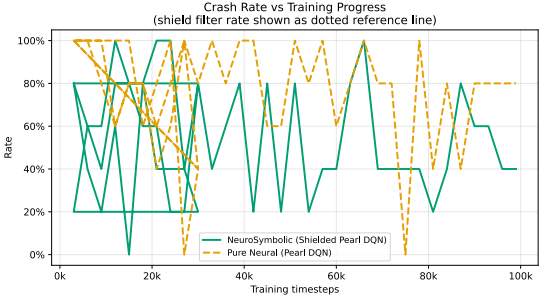


Figure: Shield override rate during training

- **Neural** reward rises faster, but unconstrained exploration yields more crashes
- Shield **override rate decreases** over training: policy learns to avoid vetoes
- Shielded agent reaches a **lower steady-state crash rate** despite slower initial reward growth

Results: Training Dynamics

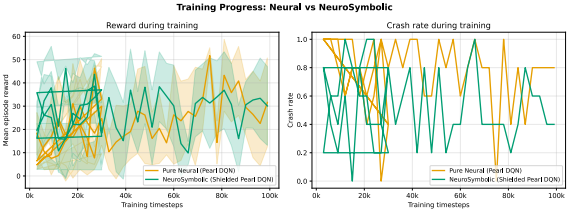


Figure: Training reward & crash rate vs. timestep (100k steps)

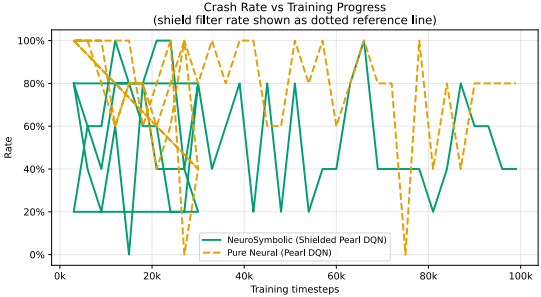


Figure: Shield override rate during training

- **Neural** reward rises faster, but unconstrained exploration yields more crashes
- Shield **override rate decreases** over training: policy learns to avoid vetoes
- Shielded agent reaches a **lower steady-state crash rate** despite slower initial reward growth

Results: Training Dynamics

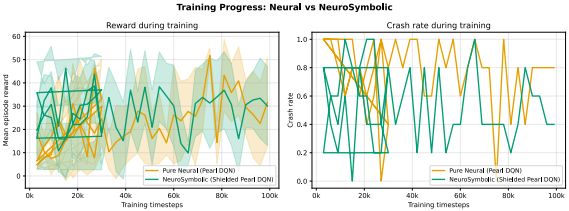


Figure: Training reward & crash rate vs. timestep (100k steps)

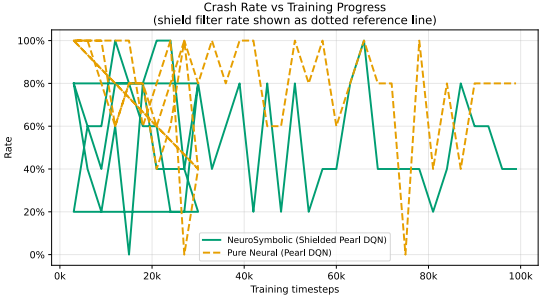


Figure: Shield override rate during training

- **Neural** reward rises faster, but unconstrained exploration yields more crashes
- Shield **override rate decreases** over training: policy learns to avoid vetoes
- Shielded agent reaches a **lower steady-state crash rate** despite slower initial reward growth

Results: 100-Episode Evaluation

Figure: Reward / speed / crash rate comparison

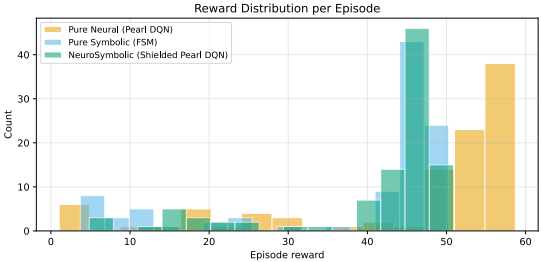


Figure: Reward distribution across 100 episodes

Agent	Reward ↑	Speed ↑	Crash ↓	Std ↓
Pure Neural	45.7	27.4 m/s	27%	16.3
Pure Symbolic (FSM)	38.3	23.0 m/s	25%	14.8
Neuro-Symbolic	**40.8**	23.0 m/s	**18%**	**11.4**

Results: 100-Episode Evaluation

Figure: Reward / speed / crash rate comparison

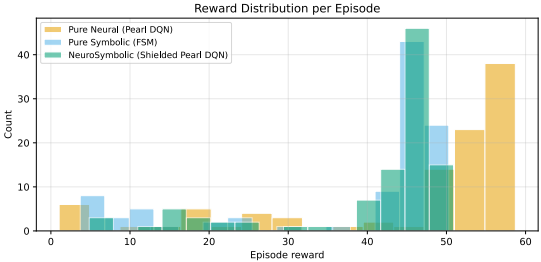


Figure: Reward distribution across 100 episodes

Agent	Reward ↑	Speed ↑	Crash ↓	Std ↓
Pure Neural	45.7	27.4 m/s	27%	16.3
Pure Symbolic (FSM)	38.3	23.0 m/s	25%	14.8
Neuro-Symbolic	**40.8**	23.0 m/s	**18%**	**11.4**

Results: Controlling for Spawn-Induced Crashes

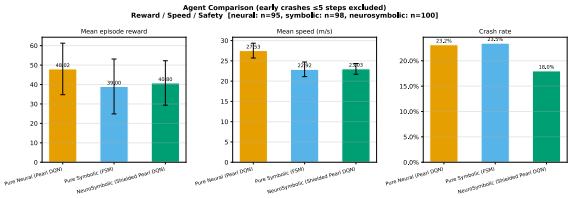


Figure: Episodes with crash in ≤ 5 steps removed

Why filter?

- HighwayEnv can spawn vehicles directly adjacent to ego
- Crash $\leq$5 steps: *environment artifact*, not policy failure
- Filtering isolates *policy-attributable* crashes

Agent	n	Crash Rate
Neural	95	23.2%
Symbolic	98	23.5%
Neuro-Sym.	**100**	**18.0%**

- Only Neuro-Sym. has **zero** spawn crashes; the shield blocks proximity collisions too
- FSM *equals* neural after filtering; pure rules add nothing without learning
- **5.2 pp** gap: attributable to the combined policy-shield effect

Results: Controlling for Spawn-Induced Crashes

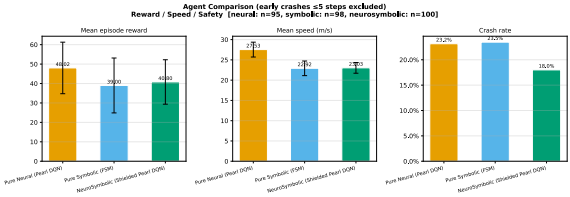


Figure: Episodes with crash in ≤ 5 steps removed

Why filter?

- HighwayEnv can spawn vehicles directly adjacent to ego
- Crash $\leq$5 steps: *environment artifact*, not policy failure
- Filtering isolates *policy-attributable* crashes

Agent	n	Crash Rate
Neural	95	23.2%
Symbolic	98	23.5%
Neuro-Sym.	**100**	**18.0%**

- Only Neuro-Sym. has **zero** spawn crashes; the shield blocks proximity collisions too
- FSM *equals* neural after filtering; pure rules add nothing without learning
- **5.2 pp** gap: attributable to the combined policy-shield effect

Results: Controlling for Spawn-Induced Crashes

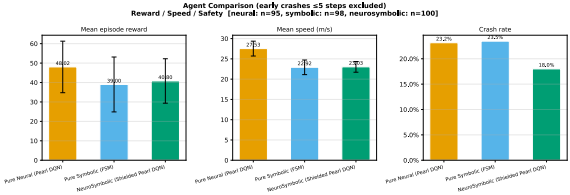


Figure: Episodes with crash in ≤ 5 steps removed

Why filter?

- HighwayEnv can spawn vehicles directly adjacent to ego
- Crash $\leq$5 steps: *environment artifact*, not policy failure
- Filtering isolates *policy-attributable* crashes

Agent	n	Crash Rate
Neural	95	23.2%
Symbolic	98	23.5%
Neuro-Sym.	**100**	**18.0%**

- Only Neuro-Sym. has **zero** spawn crashes; the shield blocks proximity collisions too
- FSM *equals* neural after filtering; pure rules add nothing without learning
- **5.2 pp** gap: attributable to the combined policy-shield effect

Results: Controlling for Spawn-Induced Crashes

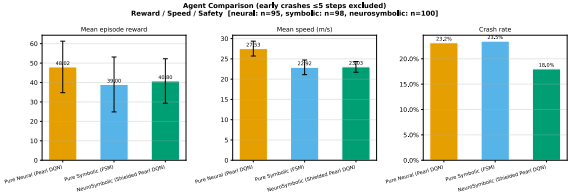


Figure: Episodes with crash in ≤ 5 steps removed

Why filter?

- HighwayEnv can spawn vehicles directly adjacent to ego
- Crash $\leq$5 steps: *environment artifact*, not policy failure
- Filtering isolates *policy-attributable* crashes

Agent	n	Crash Rate
Neural	95	23.2%
Symbolic	98	23.5%
Neuro-Sym.	**100**	**18.0%**

- Only Neuro-Sym. has **zero** spawn crashes; the shield blocks proximity collisions too
- FSM *equals* neural after filtering; pure rules add nothing without learning
- **5.2 pp** gap: attributable to the combined policy-shield effect

Results: Controlling for Spawn-Induced Crashes

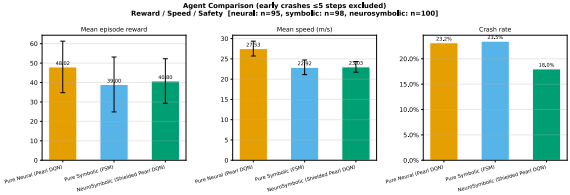


Figure: Episodes with crash in ≤ 5 steps removed

Why filter?

- HighwayEnv can spawn vehicles directly adjacent to ego
- Crash $\leq$5 steps: *environment artifact*, not policy failure
- Filtering isolates *policy-attributable* crashes

Agent	n	Crash Rate
Neural	95	23.2%
Symbolic	98	23.5%
Neuro-Sym.	**100**	**18.0%**

- Only Neuro-Sym. has **zero** spawn crashes; the shield blocks proximity collisions too
- FSM *equals* neural after filtering; pure rules add nothing without learning
- **5.2 pp** gap: attributable to the combined policy-shield effect

Discussion & Conclusions

What Worked

- Environment-level shielding ensures correct Bellman targets
- DeepSet handles permutation variance; policy focuses on strategy
- Override penalty measurably incentivizes safety internalization
- Best safety-consistency trade-off across all three agents

Limitations & Future Work

- Reward gap: ~5 pts lost to conservative shielding; adaptive thresholds could close this
- Spawn crashes: need multi-step lookahead or a probabilistic shield
- Learned specs: neural-guided LTL synthesis is an open direction
- Scaling: continuous action spaces require differentiable shields

Bottom Line

Combining a **learned DeepSet Q-network** with a **formal LTL safety shield** reduces crash rate by **33%** and reward variance by **30%** versus pure neural; each blocked action is traceable to a named rule.

Discussion & Conclusions

What Worked

- Environment-level shielding ensures correct Bellman targets
- DeepSet handles permutation variance; policy focuses on strategy
- Override penalty measurably incentivizes safety internalization
- Best safety-consistency trade-off across all three agents

Limitations & Future Work

- Reward gap: ∼5 pts lost to conservative shielding; adaptive thresholds could close this
- Spawn crashes: need multi-step lookahead or a probabilistic shield
- Learned specs: neural-guided LTL synthesis is an open direction
- Scaling: continuous action spaces require differentiable shields

Bottom Line

Combining a **learned DeepSet Q-network** with a **formal LTL safety shield** reduces crash rate by **33%** and reward variance by **30%** versus pure neural; each blocked action is traceable to a named rule.

Discussion & Conclusions

What Worked

- Environment-level shielding ensures correct Bellman targets
- DeepSet handles permutation variance; policy focuses on strategy
- Override penalty measurably incentivizes safety internalization
- Best safety-consistency trade-off across all three agents

Limitations & Future Work

- Reward gap: $\sim$5 pts lost to conservative shielding; adaptive thresholds could close this
- Spawn crashes: need multi-step lookahead or a probabilistic shield
- Learned specs: neural-guided LTL synthesis is an open direction
- Scaling: continuous action spaces require differentiable shields

Bottom Line

Combining a **learned DeepSet Q-network** with a **formal LTL safety shield** reduces crash rate by **33%** and reward variance by **30%** versus pure neural; each blocked action is traceable to a named rule.

Discussion & Conclusions

What Worked

- Environment-level shielding ensures correct Bellman targets
- DeepSet handles permutation variance; policy focuses on strategy
- Override penalty measurably incentivizes safety internalization
- Best safety-consistency trade-off across all three agents

Limitations & Future Work

- Reward gap: $\sim$5 pts lost to conservative shielding; adaptive thresholds could close this
- Spawn crashes: need multi-step lookahead or a probabilistic shield
- Learned specs: neural-guided LTL synthesis is an open direction
- Scaling: continuous action spaces require differentiable shields

Bottom Line

Combining a **learned DeepSet Q-network** with a **formal LTL safety shield** reduces crash rate by **33%** and reward variance by **30%** versus pure neural; each blocked action is traceable to a named rule.

Discussion & Conclusions

What Worked

- Environment-level shielding ensures correct Bellman targets
- DeepSet handles permutation variance; policy focuses on strategy
- Override penalty measurably incentivizes safety internalization
- Best safety-consistency trade-off across all three agents

Limitations & Future Work

- Reward gap: $\sim$5 pts lost to conservative shielding; adaptive thresholds could close this
- Spawn crashes: need multi-step lookahead or a probabilistic shield
- Learned specs: neural-guided LTL synthesis is an open direction
- Scaling: continuous action spaces require differentiable shields

Bottom Line

Combining a **learned DeepSet Q-network** with a **formal LTL safety shield** reduces crash rate by **33%** and reward variance by **30%** versus pure neural; each blocked action is traceable to a named rule.

Discussion & Conclusions

What Worked

- Environment-level shielding ensures correct Bellman targets
- DeepSet handles permutation variance; policy focuses on strategy
- Override penalty measurably incentivizes safety internalization
- Best safety-consistency trade-off across all three agents

Limitations & Future Work

- Reward gap: $\sim$5 pts lost to conservative shielding; adaptive thresholds could close this
- Spawn crashes: need multi-step lookahead or a probabilistic shield
- Learned specs: neural-guided LTL synthesis is an open direction
- Scaling: continuous action spaces require differentiable shields

Bottom Line

Combining a **learned DeepSet Q-network** with a **formal LTL safety shield** reduces crash rate by **33%** and reward variance by **30%** versus pure neural; each blocked action is traceable to a named rule.

Discussion & Conclusions

What Worked

- Environment-level shielding ensures correct Bellman targets
- DeepSet handles permutation variance; policy focuses on strategy
- Override penalty measurably incentivizes safety internalization
- Best safety-consistency trade-off across all three agents

Limitations & Future Work

- Reward gap: ∼5 pts lost to conservative shielding; adaptive thresholds could close this
- Spawn crashes: need multi-step lookahead or a probabilistic shield
- Learned specs: neural-guided LTL synthesis is an open direction
- Scaling: continuous action spaces require differentiable shields

Bottom Line

Combining a **learned DeepSet Q-network** with a **formal LTL safety shield** reduces crash rate by **33%** and reward variance by **30%** versus pure neural; each blocked action is traceable to a named rule.